

DSAP

Laboratory Manual

LAB 02

Discrete-Time Signal Generation and Manipulation in MATLAB

	DSAP
Lab Title	Discrete-Time Signal Generation and Manipulation in MATLAB
Prerequisites	Basic programming concepts
Software	MATLAB R2020a or later

1. Lab Objectives

By the end of this lab, students will be able to:

- Generate and visualise discrete-time unit impulse and unit step signals in MATLAB.
- Write reusable MATLAB functions (impseq, stepseq, sigadd, sigshift, sigfold, sigmult).
- Perform arithmetic operations — addition, scaling, shifting, folding, and multiplication — on discrete signals.
- Construct complex sequences by combining elementary signals.
- Interpret stem plots and relate graphical output to mathematical definitions.
- Debug MATLAB code and verify results analytically.

Part A — Discrete-Time Unit Impulse Signal

Background

The discrete-time unit impulse (Dirac delta) is defined as:

$$\delta[n] = 1 \quad \text{if } n = 0, \quad 0 \quad \text{otherwise}$$

A shifted impulse $\delta[n - n_0]$ equals 1 only when $n = n_0$. In MATLAB, the logical expression $(n == n_0)$ exploits the fact that MATLAB returns 1 for true and 0 for false, giving a clean one-line implementation.

Helper Function: impseq

Create a file impseq.m with the following content:

```
function [x, n] = impseq(n0, n1, n2)
% Generates x(n) = delta(n - n0) over n1 <= n <= n2
% Usage: [x, n] = impseq(n0, n1, n2)
    n = [n1 : n2];
    x = [(n - n0) == 0];
end
```

Task 1: Basic Impulse Signals

Interval: $-5 \leq n \leq 10$

Generate and plot each of the following impulse sequences using impseq.

- Plot $\delta[n]$ (impulse centred at $n = 0$)
- Plot $\delta[n - 4]$ (impulse shifted right by 4 samples)
- On the same figure, use subplot to display both plots side-by-side
- Label axes, add titles, and enable grid

Starter code for part (a):

```
clc; clear all; close all;
n = -5 : 1 : 10;
[delta, n] = impseq(0, -5, 10); % delta[n]
subplot(1, 2, 1);
stem(n, delta, 'k', 'filled');
xlabel('n'); ylabel('Amplitude'); title('δ[n]'); grid on;
% Add subplot(1,2,2) for delta[n-4] here
```

Task 2: Scaled Impulse Combination

Interval: $-5 \leq n \leq 5$

Using `impseq`, generate and plot the composite signal below over $-5 \leq n \leq 5$.

- $x(n) = 2\delta(n + 2) - \delta(n - 4)$
- Verify analytically: list every non-zero sample value before running the code
- Mark the non-zero samples on your plot using `text()` or a different colour stem

Part B — Discrete-Time Unit Step Signal

Background

The discrete-time unit step is defined as:

$$u[n] = 1 \quad \text{if } n \geq 0, \quad 0 \quad \text{otherwise}$$

Helper Function: `stepseq`

Create a file `stepseq.m` with the following content:

```
function [x, n] = stepseq(n0, n1, n2)
% Generates x(n) = u(n - n0) over n1 <= n <= n2
% Usage: [x, n] = stepseq(n0, n1, n2)
n = [n1 : n2];
x = [(n - n0) >= 0];
end
```

Task 3: Step Signal Plots

Interval: $-5 \leq n \leq 10$

Generate and plot the following step sequences, each on a separate subplot.

- $u[n]$ — standard unit step
- $u[n - 3]$ — step delayed by 3 samples

Task 4: Piecewise Exponential-Linear Sequence*Interval: $0 \leq n \leq 20$*

Generate and plot the composite sequence below over $0 \leq n \leq 20$. This models a signal that rises linearly then decays exponentially after $n = 10$.

- $x(n) = n[u(n) - u(n-10)] + 10e^{(-0.3(n-10))}[u(n-10) - u(n-20)]$
- Add a vertical dashed line at $n = 10$ using `xline(10, '--r', 'n=10')`
- Label the two regions on the plot: 'Linear rise' and 'Exponential decay'
- What is the value of $x(10)$? Verify by hand.

Partial starter code:

```
n = [0:20];
x1 = n .* (stepseq(0,0,20) - stepseq(10,0,20));
x2 = 10*exp(-0.3*(n-10)) .* (stepseq(10,0,20) - stepseq(20,0,20));
x = x1 + x2;
% Add stem plot, labels, and xline here
```

Part C — Signal Operations

Helper Functions to Create

Before starting the tasks, create the following four function files. They will be reused across this and future labs.

sigadd — Signal Addition

```
function [y, ny] = sigadd(x1, n1, x2, n2)
% General addition y(n) = x1(n) + x2(n) with different support
ny = min(min(n1), min(n2)) : max(max(n1), max(n2));
y1 = zeros(1, length(ny)); y2 = zeros(1, length(ny));
y1((ny >= min(n1)) & (ny <= max(n1))) = x1;
y2((ny >= min(n2)) & (ny <= max(n2))) = x2;
y = y1 + y2;
end
```

sigshift — Signal Shifting

```
function [y, ny] = sigshift(x, n, n0)
% Shift: y(n) = x(n - n0)
y = x; ny = n + n0;
end
```

sigfold — Signal Folding

```
function [y, n] = sigfold(x, n)
% Fold: y(n) = x(-n)
y = fliplr(x); n = -fliplr(n);
end
```

sigmult — Signal Multiplication

```
function [y, ny] = sigmult(x1, n1, x2, n2)
% Element-wise multiplication on union support
ny = min(min(n1),min(n2)) : max(max(n1),max(n2));
y1 = zeros(1,length(ny)); y2 = zeros(1,length(ny));
y1((ny>=min(n1)) & (ny<=max(n1))) = x1;
y2((ny>=min(n2)) & (ny<=max(n2))) = x2;
y = y1 .* y2;
end
```

⚠ Common Pitfall

When using element-wise operators (`.*`, `./`), make sure both arrays are the same size. Use `sigadd`/`sigmult` — which handle mismatched supports — rather than direct arithmetic on raw arrays.

Task 5: Shifting and Combination

Interval: $-7 \leq n \leq 15$ (expected output range)

Let $x(n) = \{1, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2, 1\}$ defined over $-2 \leq n \leq 10$. Generate and plot $x_1(n) = 2x(n-5) - 3x(n+4)$.

- First define: $n = -2:10$; $x = [1:7, 6:-1:1]$;
- Use `sigshift` to obtain $2x(n-5)$ and $-3x(n+4)$ separately
- Use `sigadd` to combine them
- Verify: what is $x_1(0)$?

Task 6: Folding and Multiplication

Interval: $-7 \leq n \leq 12$

Using the same $x(n)$ from Task 5, compute and plot $x_2(n) = x(3-n) + x(n)x(n-2)$.

- Note: $x(3-n) = x(-(n-3))$ — fold first, then shift by 3
- Compute $x(n)x(n-2)$ using `sigmult` (mind the support alignment)
- Use `sigadd` to combine the two terms
- Plot $x_2(n)$; your result should match the reference plot in `Lab02_Draft.pdf`

Part D — Combination of multiple concepts**Task 7: Windowed Sinusoidal Signal**

Interval: $0 \leq n \leq 40$

Generate and plot the following windowed discrete sinusoid over $-0 \leq n \leq 40$:

- $x(n) = \cos(0.1\pi n) \cdot [u(n) - u(n-20)] + 0.5\sin(0.2\pi n) \cdot [u(n-20) - u(n-40)]$
- Use `stepseq` for all step functions
- Plot using `stem`; add a stem plot of $\cos(0.1\pi n)$ for reference on a second subplot

- Describe in a comment: what does windowing do to the sinusoid?

Task 8: Energy and Power Computation*Interval: General function*

Write a MATLAB function `energypower.m` that accepts a signal array `x` and returns its energy `E` and average power `P` over the given window.

- $E = \sum |x(n)|^2$ over all provided samples
- $P = E / N$ where $N = \text{length}(x)$
- Apply this function to the signals from Tasks 5, 6, and 7
- Display results in a formatted table using `fprintf`
- Bonus: Plot the energy contributions per sample ($|x(n)|^2$) for Task 7

Function signature to complete:

```
function [E, P] = energypower(x)
% Computes energy and average power of discrete signal x
% Your code here
end
```

Quick Reference — MATLAB Syntax List

Task	MATLAB Code
Create time vector (continuous)	<code>t = 0 : 0.001 : 1;</code>
Create discrete index	<code>n = 0 : 1 : 50;</code>
Evenly spaced points	<code>t = linspace(0, 1, 1000);</code>
Generate sine wave	<code>x = sin(2*pi*f*t);</code>
Generate cosine wave	<code>x = cos(2*pi*f*t);</code>
Unit step (continuous)	<code>u = (t >= 0);</code>
Unit step (discrete)	<code>u = (n >= 0);</code>
Unit impulse (discrete)	<code>delta = (n == 0);</code>
Ramp signal	<code>r = t .* (t >= 0);</code>
Sinc function	<code>x = sinc(t);</code>
Decaying exponential	<code>x = exp(-a*t);</code>
Complex exponential	<code>x = exp(1j * omega * t);</code>
Real part	<code>real(x)</code>
Imaginary part	<code>imag(x)</code>
Magnitude	<code>abs(x)</code>
Continuous plot	<code>plot(t, x, 'b', 'LineWidth', 1.5);</code>
Discrete plot	<code>stem(n, x, 'filled');</code>
Multiple subplots	<code>subplot(rows, cols, index);</code>
Add title	<code>title('Signal Name');</code>
Add x label	<code>xlabel('Time (s)');</code>
Add y label	<code>ylabel('Amplitude');</code>
Turn grid on	<code>grid on;</code>
Set axis limits	<code>axis([xmin xmax ymin ymax]);</code>
Add legend	<code>legend('Signal 1', 'Signal 2');</code>